

1.

Tienes un framework que permite conectar distintas bases de datos (PostgreSQL, MySQL, Oracle, etc.) y quieres que el **código cliente pueda cambiar de base de datos sin modificar su lógica de negocio**. Además, cada base de datos necesita crear distintos tipos de objetos de acceso a datos (DAO, conexiones, transacciones).

¿Cuál patrón creacional encaja mejor en este escenario?

- A. Factory Method
 - B. Abstract Factory
 - C. Builder
 - D. Prototype
-

2.

En un sistema de reservas de vuelos, cada vez que se crea una nueva instancia de “Vuelo Charter” se debe aplicar una **configuración estándar compleja** (tarifas, políticas, slots de horarios, servicios incluidos).

Crear estos objetos manualmente es costoso en tiempo y propenso a errores.

¿Qué patrón creacional sería más adecuado?

- A. Abstract Factory
 - B. Builder
 - C. Prototype
 - D. Singleton
-

3.

En una librería de gráficos 3D, la construcción de un objeto `Escena` implica configurar múltiples elementos opcionales (luces, sombras, texturas, cámaras, efectos especiales) en un orden controlado. Además, podrían existir **constructores especializados** (por ejemplo: escenas realistas vs escenas de prueba).

¿Qué patrón aplicarías para mantener **flexibilidad y claridad** en la creación de escenas?

- A. Builder
 - B. Factory Method
 - C. Singleton
 - D. Abstract Factory
-

4.

Estás diseñando una API para un motor de juegos. Necesitas asegurarte de que **solo haya una instancia global de la clase `GameEngine`**, pero además, quieres que la creación de esa instancia pueda **delegarse dinámicamente a diferentes inicializadores**

(por ejemplo, un inicializador en modo test, otro en modo producción).
¿Qué combinación de patrones creacionales es más adecuada?

- A. Singleton + Builder
 - B. Singleton + Abstract Factory
 - C. Singleton + Prototype
 - D. Factory Method + Prototype
-

5.

Un sistema de personalización de productos permite clonar plantillas existentes y luego modificarlas ligeramente para adaptarlas a nuevos clientes. El número de atributos es grande, y muchos tienen valores por defecto.

¿Cuál de los siguientes enfoques sería más eficiente y mantenible?

- A. Crear todos los objetos con un constructor lleno de parámetros
- B. Usar Abstract Factory para crear nuevas plantillas
- C. Clonar un objeto base con Prototype y aplicar modificaciones
- D. Usar un Singleton que devuelva la misma plantilla para todos los clientes